

The mixture-of-experts model

1. How the model works

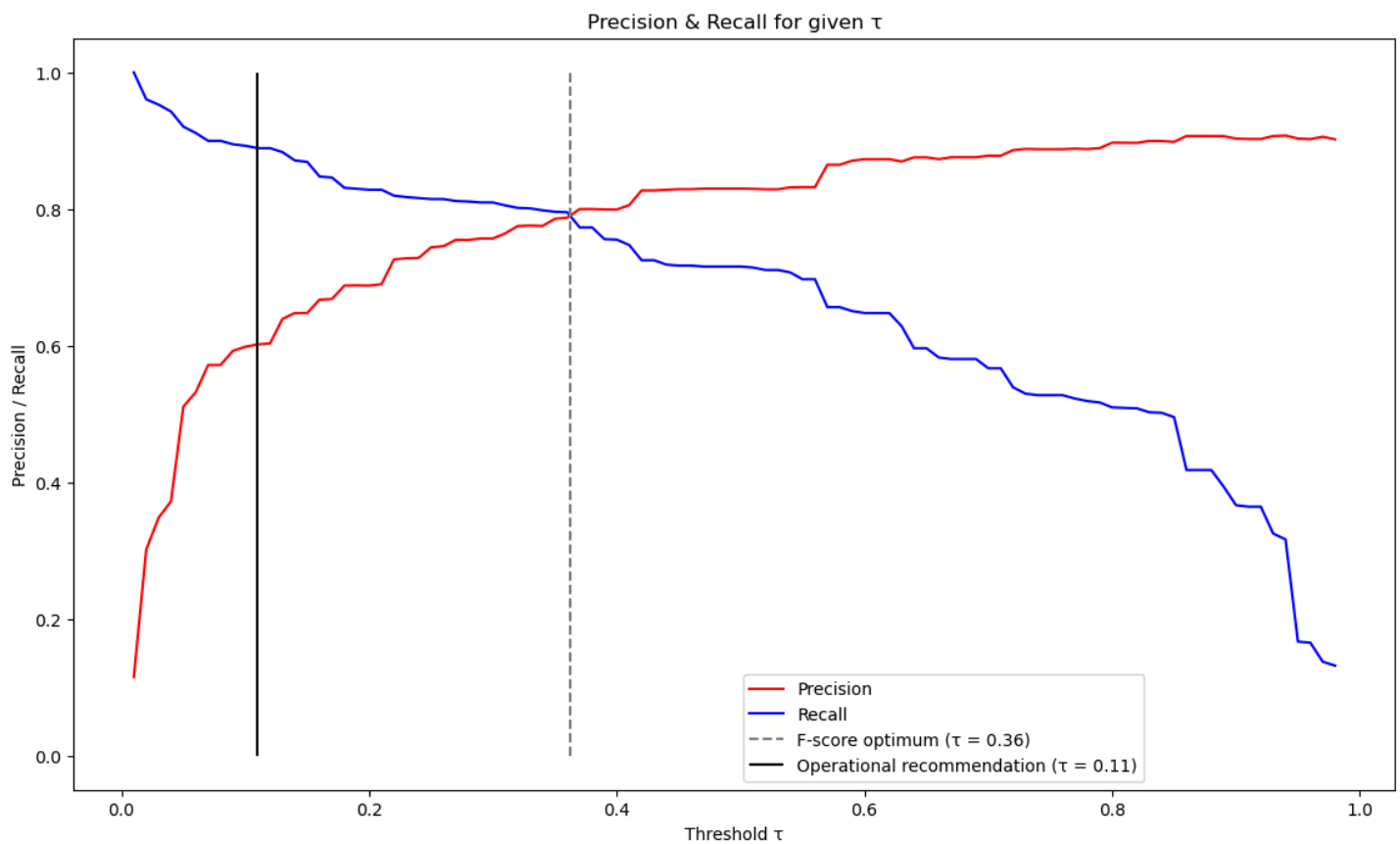
The mixture-of-experts model consists of three components: a temperature model, an accelerometer model, and an hour model. Each model uses an if-tree structure to generate a single value. These values are combined using weighted sums and transformed through a sigmoid function to calculate the probability of an animal being outside. The ublox-module should attempt a fix when this probability exceeds a user-defined threshold τ . The tau value controls the model's specificity and sensitivity—a high value results in fewer but more precise fixes (since the ublox only activates at high probabilities), while a low value captures more outside events but generates more false alarms.

$$z = bias + w_t p_t + w_a p_a + w_h p_h$$

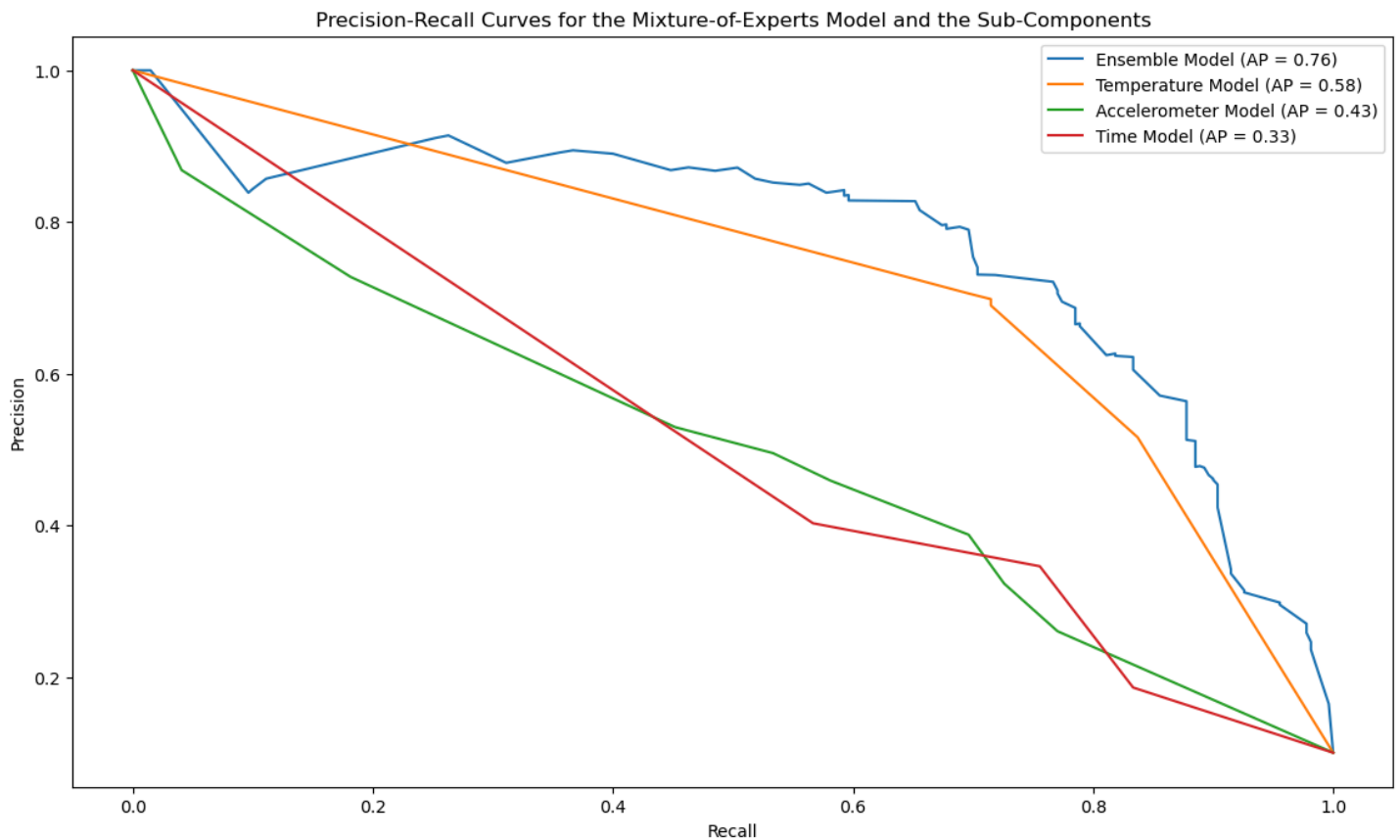
$$p = \sigma(z) = \frac{1}{(1 + e^{-z})}$$

Trigger ublox if $p \geq \tau$

The Precision-Recall Curve shows the expected performance for a given τ .



The mixture-of-experts model allows us to combine the predictive power from different signals while maintaining a modular setup, where each component can be disabled if its signal becomes unreliable (by setting its weight to zero).



2. How to implement the model

The model operates on a fixed schedule, making a decision at regular intervals—such as every hour—on whether or not to initiate a GPS fix. This decision is based on sensor readings collected since the last scheduled fix and compares them against statistical aggregates from previous intervals. Rather than relying on raw sensor values, the model uses aggregated features (e.g., means, sums, temporal differences, and rolling statistics) to detect patterns indicative of behavioral state changes.

2. 1 The temperature model

Variable	Explanation
temp_mean_dev_z_20	The deviation of the mean temperature during the current inter-fix period from the rolling mean of the previous 20 periods, normalized by the rolling standard deviation over the same 20 periods.
temp_max_z_diff	The maximum absolute deviation of temperature readings from their mean within the current inter-fix period, divided by the standard deviation of those deviations (i.e., the within-period variation of absolute differences).

```
|--- temp_mean_dev_z_20 <= -1.46
| |--- temp_max_z_diff <= 1.40
| | |--- value: [0.72]
| |--- temp_max_z_diff > 1.40
| | |--- value: [0.38]
|--- temp_mean_dev_z_20 > -1.46
| |--- temp_mean_dev_z_20 <= -0.87
| | |--- value: [0.27]
| |--- temp_mean_dev_z_20 > -0.87
| | |--- temp_mean_dev_z_20 <= 2.74
| | | |--- value: [0.03]
| | |--- temp_mean_dev_z_20 > 2.74
| | | |--- value: [0.50]
```

Implementation notes in Python

```
def max_z_diff(x):
    numerator = (x.diff().abs().max() - x.diff().abs().mean())
    denominator = (x.diff().abs().std(ddof=0) + 1e-6) # 1e-6 as numerical safeguard
    return numerator / denominator
```

2.2 The accelerometer model

The magnitude equals the length of the movement vector, i.e., $\sqrt{acc_x^2 + acc_y^2 + acc_z^2}$

Variable	Explanation
acc_y+z_mean	The mean of the sum of acceleration along the Y and Z axes during the current inter-fix period.
acc_x-z_max	The maximum of the difference between X- and Z-axis acceleration in the current inter-fix period.
acc_x-z_mean	The average difference between X- and Z-axis acceleration.
mag_var	The variance of the acceleration vector magnitude during the fix period.
acc_x-z_sum	The total cumulative difference between X and Z acceleration across the fix period

```
|--- acc_y+z_mean <= -9.87
| |--- acc_x-z_max <= 9.02
| | |--- acc_x-z_mean <= 5.39
| | | |--- value: [0.10]
| | |--- acc_x-z_mean > 5.39
| | | |--- mag_var <= 0.23
| | | | |--- value: [0.33]
| | | |--- mag_var > 0.23
| | | | |--- value: [0.88]
| |--- acc_x-z_max > 9.02
| | |--- mag_var <= 0.20
| | | |--- value: [0.57]
| | |--- mag_var > 0.20
| | | |--- value: [0.83]
|--- acc_y+z_mean > -9.87
| |--- acc_x-z_mean <= 6.96
| | |--- value: [0.02]
| |--- acc_x-z_mean > 6.96
| | |--- acc_x-z_sum <= 13.73
| | | |--- value: [0.48]
| | |--- acc_x-z_sum > 13.73
| | | |--- mag_var <= 0.16
| | | | |--- value: [0.09]
| | | |--- mag_var > 0.16
| | | | |--- value: [0.33]
```

Implementation notes

```
axes = ["x", "y", "z"]
data["mag"] = np.sqrt(data[[f"acc_{a}" for a in axes]].pow(2).sum(axis=1))
for i, j in it.combinations(axes, 2):
    data[f"acc_{i}+{j}"] = data[f"acc_{i}"] + data[f"acc_{j}"]
    data[f"acc_{i}-{j}"] = data[f"acc_{i}"] - data[f"acc_{j}"]

# And then aggregate through groupby(["DEPLOYMENT_ID", "fix_cycle"]).agg("sum", ...)
```

2.3 The hour model

$$\text{sine encoding of the hour} = \sin\left(\frac{2\pi \times \text{hour}}{24}\right)$$

Variable	Explanation
hour_sin	A cyclical encoding of the hour of day using the sine function.
hour_cos	A cyclical encoding of the hour of day using the cosine function.

```
|--- hour_sin <= 0.00
| |--- hour_cos <= -0.13
| | |--- value: [0.07]
| |--- hour_cos > -0.13
| | |--- hour_cos <= 0.92
| | | |--- value: [0.42]
| | |--- hour_cos > 0.92
| | | |--- value: [0.21]
|--- hour_sin > 0.00
| |--- value: [0.02]
```

Implementation notes

```
aggregated["hour"] = aggregated["Timestamp"].dt.hour
aggregated['hour_sin'] = np.sin(2 * np.pi * aggregated['hour'] / 24)
aggregated['hour_cos'] = np.cos(2 * np.pi * aggregated['hour'] / 24)
```

2.4 The ensemble

Each decision tree produces a value. These values are multiplied by their respective weights, summed together with a bias term, and transformed through a sigmoid function to calculate the final probability.

```
value = bias + w_H * H_pred + w_A * A_pred + w_T * T_pred
pred = 1 / (1 + np.exp(-value))
if pred >= tau:
    print("Yes, let's attempt to get a GPS fix.")
else:
    print("No, let's skip this interval.")
```

The default weights and bias were determined through logistic regression on the training data and validated on the test set. If you need to remove a component, set its weight to zero and recalculate the weights and bias.

Weight	Default Value
Temperature model	4.715
Accelerometer model	4.906
Hour model	3.961
Bias	-4.917

Tau is chosen to balance data value with battery life.

Default value	Explanation
0.36	Optimizes the F1-Score, providing the best balance between recall (how many true fixes are captured) and precision (how many predicted fixes are correct). This setting offers a good trade-off between performance and battery conservation.
0.11	Operational recommendation when high recall is essential. At this threshold, the model captures ~89% of all true fixes, even if it means accepting a lower precision (~51%). Suitable when missing a fix is more costly than attempting one unnecessarily.

3. Components to keep configurable

Component	Default Value / Explanation	Priority
Threshold tau	Essential to adapt the model to different trade-offs between precision and recall.	Critical
Intercept & Weights	Crucial for enabling modular ensemble logic and combining sub-models effectively.	Critical
Temperature Features	Recommended to keep adjustable due to potential seasonal variation in temperature dynamics.	Necessary
Hour features	Advisable to retain as configurable, since changing day length may alter pangolin activity rhythms.	Good to have

Keeping the features configurable presents some complexity. Rather than coding individual variables (like "temp_mean_dev_z_20_first_split_thres"), a better approach would be to load the entire tree structure from a JSON file in this format:

```
{
  "feature_map": {
    "temp_mean_dev_z_20": 0,
    "temp_max_z_diff": 1
  },
  "nodes": [
    { "feat": 0, "th": -1.46, "l": 1, "r": 4 },           // node 0
    { "feat": 1, "th": 1.40, "l": 2, "r": 3 },           // node 1
    { "leaf": 0.72 },                                   // node 2
    { "leaf": 0.38 },                                   // node 3
    { "feat": 0, "th": -0.87, "l": 5, "r": 6 },           // node 4
    { "leaf": 0.27 },                                   // node 5
    { "feat": 0, "th": 2.74, "l": 7, "r": 8 },           // node 6
    { "leaf": 0.03 },                                   // node 7
    { "leaf": 0.50 }                                     // node 8
  ]
}
```

After loading the JSON for each tree, the code traverses the tree structure and returns the corresponding value.

```
def eval_temp(nodes, feats):
    idx = 0
    while idx not None:
        node = nodes[idx]
        if "leaf" in node:
            return node["leaf"]
        idx = node["l"] if feats[node["feat"]] <= node["th"] else node["r"]
```

I realize this adds considerable implementation overhead, but it could lay important groundwork for future on-device AI applications — such as classifying behavior from high-frequency accelerometer data over the past 15 minutes, storing the result, and then clearing the buffer. That said, if it's too much hassle at this stage, please feel free to omit it.